3.1 Front-end (8pt)

1. Utilizzo HTML/CSS/JS

Nel progetto ho utilizzato in modo concreto HTML, CSS e JavaScript tramite React, una libreria JavaScript che impiega la sintassi JSX. La struttura dell'interfaccia è organizzata in componenti semantici e riutilizzabili, mentre la parte stilistica combina Tailwind CSS, utilizzato attraverso le classi nei vari elementi, con fogli di stile personalizzati, così da ottenere sia rapidità di sviluppo sia un buon livello di controllo grafico.

JavaScript non è stato impiegato solo per la visualizzazione degli elementi, ma anche per la gestione dell'aggiornamento dei dati, delle interazioni utente, del cambio tema, dell'esportazione in CSV e del comportamento dinamico della dashboard.

Ho utilizzato inoltre la libreria Axios, che consente di gestire le richieste HTTP tramite i principali metodi CRUD, come .get(), .put() e .post(). In questo modo il front-end comunica con Laravel attraverso l'API, che a sua volta dialoga con il server. Questa comunicazione non è solo in ascolto: nel caso della ventola, ad esempio, vengono inviati comandi diretti per determinarne lo stato di accensione o spegnimento, oltre alla modalità di funzionamento, manuale oppure automatica.

2. Qualità grafica

La qualità grafica è uno degli aspetti su cui ho lavorato maggiormente, anche in collaborazione con Andrea De Togni.

Ho scelto uno stile tecnico e coerente con il tema smart home e stazione meteo, definendo palette dedicate per ciascun sensore, card leggibili, grafici armonizzati, animazioni leggere e supporto al tema chiaro/scuro. Il risultato è un'interfaccia moderna, riconoscibile e curata, in cui ogni colore è stato scelto in funzione del suo ruolo.

3. Usabilità

La dashboard è stata progettata per risultare intuitiva anche al primo utilizzo. Le informazioni più importanti sono posizionate nella parte alta e quindi immediatamente visibili, mentre le azioni operative, come esportazione, gestione account, cambio tema e controllo della ventola, sono raccolte in aree chiare dell'interfaccia.

Ogni sezione ha una funzione precisa e l'utente non deve cercare i dati, perché li trova già organizzati in modo logico. In questo senso, React è stato particolarmente utile perché mi ha permesso di collegare i vari componenti del sito tramite il sistema di import ed export, mantenendo la struttura ordinata e modulare.

4. Responsive design

L'interfaccia è progettata per adattarsi a schermi di dimensioni diverse. Le card dei sensori cambiano disposizione in base alla larghezza disponibile, i grafici utilizzano contenitori responsive e anche header e pannelli si riorganizzano correttamente tra desktop e mobile. Questo consente di mantenere leggibilità e usabilità senza dover sviluppare un'interfaccia separata.

```
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
  {cards.map((card) => (
    <SensorCard key={card.label} {...card} />  ))}
</div>
```

5. Chiarezza informativa

Ogni valore mostrato è accompagnato da etichetta, unità di misura, colore coerente e posizione riconoscibile. Ho evitato di sovraccaricare la pagina con troppe informazioni nello stesso momento: prima vengono presentati i dati principali, poi lo storico e infine i controlli.

Questa scelta aiuta l’utente a comprendere subito ciò che sta leggendo e riduce il rischio di confusione. All’inizio della pagina sono presenti l’orario e le card con i dati specifici; sotto si trovano i grafici interattivi; nella parte finale, invece, sono raccolte le funzionalità operative disponibili.

3.2 Framework e librerie (6pt)

1. Corretto utilizzo framework

Nel progetto non ho utilizzato un framework completo, ma ho fatto uso di librerie e strumenti specifici per il front-end:

React, libreria JavaScript per la costruzione dell’interfaccia

Axios, libreria JavaScript per la gestione delle richieste HTTP

Tailwind CSS, framework CSS che consente di applicare lo stile direttamente nei componenti tramite classi utility, in modo rapido, dinamico e responsive

Ho utilizzato React in modo corretto, separando logica, interfaccia e dati invece di concentrare tutto in un unico file. I componenti ricevono props, gli hook gestiscono il comportamento, mentre la pagina principale coordina le diverse sezioni.

3 esempi concreti di Tailwind CSS

Layout responsive delle card sensori in SensorCardsSection.jsx (linea 58):

```
className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4 mb-12"
```

In questo caso Tailwind crea una griglia responsive: grid-cols-1 su mobile, md:grid-cols-2 su tablet e lg:grid-cols-3 su schermi più grandi.

Input del form in AuthField.jsx (linea 46):

"h-11 w-full rounded-xl border px-3 text-sm outline-none transition"

Questa classe definisce altezza, larghezza completa, bordi arrotondati, spaziatura interna, dimensione del testo e transizione visiva senza dover scrivere CSS tradizionale.

Bottone per il cambio tema in AuthFrame.jsx (linea 47):

"inline-flex items-center gap-2 rounded-full border px-3 py-2 text-xs font-semibold"

Qui Tailwind viene utilizzato per costruire un bottone già ben strutturato, con elementi allineati orizzontalmente, spazio tra icona e testo, bordo arrotondato, padding e tipografia compatta.

2. Organizzazione componenti

Il progetto è suddiviso in aree precise: pages, layout, sections, cards, charts, hooks, constants, api.

Questa organizzazione rende il codice leggibile, facile da mantenere e semplice da estendere. È una scelta ordinata, utile per costruire una base solida prima di aggiungere nuove funzionalità.

Dashboard: lettura architetturale del blocco principale

Dashboard.jsx = componente principale che coordina la dashboard

Dashboard.template.css = design system principale

Dashboard.effects.css = effetti di background e atmosfera

Questa è la gerarchia principale del blocco dashboard.

useSensorData.js = gestione dei dati dei sensori

SensorCardsSection.jsx = collegamento tra dati e card

SensorCard.jsx = visualizzazione della singola card

TimelineSection.jsx = collegamento tra storico e grafici

TimelineChart.jsx = grafico singolo

chartsConfig.js = configurazione dei grafici

sensorsConfig.js = configurazione delle card

DashboardHeader.jsx = controlli globali della pagina

BackgroundLayers.jsx = gestione estetica dello sfondo

DashboardFooter.jsx = chiusura visiva della pagina

ClockDisplay.jsx = orologio dell’interfaccia

FanControl.jsx = interfaccia di controllo della ventola

App.jsx = routing e gestione degli accessi

posts.js / layer api = contatto con i dati

Login: lettura architetturale del blocco auth

AuthFrame.jsx = struttura visiva, tema e spotlight

AuthField.jsx = rendering del singolo campo

Login.jsx = logica di accesso

Register.jsx = logica di registrazione

Questa è la gerarchia corretta del blocco autenticazione.

Aspetti importanti:

AuthField non è un campo “intelligente”, ma un renderer controllato tramite props. Questo rappresenta un punto di forza, perché mantiene il componente semplice e riutilizzabile.

AuthFrame ha un ruolo più importante di quanto possa sembrare, perché non funge solo da wrapper visivo, ma gestisce anche tema, sincronizzazione, background, spotlight e struttura generale dell’interfaccia auth.

Login.module.css e Register.module.css sembrano non rappresentare il centro del flusso attuale, anche se sono presenti nel progetto.

Auth.css appare come un sistema parallelo o legacy: è ricco di contenuti, ma non risulta chiaramente dominante rispetto ai file React attuali.

Dashboard.template.css va confrontato con l’effettivo Dashboard.css, perché il nome del file e l’import non sembrano combaciare.

Appunti sintetici:

App.jsx = routing e controllo accessi

AuthFrame.jsx = struttura auth, tema e spotlight

AuthField.jsx = renderer degli input

Login.jsx = logica del login

Register.jsx = logica della registrazione

Architettura Dashboard HomeHero
Componenti Core
Dashboard.jsx (Regista)
Coordinamento stato e dati
Gestione orologio e tema
Distribuzione props ai figli
App.jsx (Routing)
Navigazione tra pagine
Protezione rotte (Auth)
Redirect logica accesso
Logica e Dati
useSensorData.js (Cervello)
Stato real-time e storico
Gestione export CSV
Simulazione o Fetching dati
Configurazioni (Dichiarative)
chartsConfig.js (Grafici)
sensorsConfig.js (Card)
Layer API
posts.js (Chiamate HTTP)
Centralizzazione Axios
Sezioni UI
SensorCardsSection
Mappatura dati su SensorCard
Griglia responsive
TimelineSection
Organizzatore grafici storici
Utilizzo TimelineChart
Header e Footer
DashboardHeader (Controlli)
DashboardFooter (Firma statica)
FanControl
Controllo manuale/auto ventola
Setting temperature
Presentazione e Stile
Componenti Atomici
SensorCard.jsx (Puro)
TimelineChart.jsx (Recharts)
ClockDisplay.jsx (Visivo)
Design System (CSS)
Dashboard.template.css (Struttura)
Dashboard.effects.css (Atmosfera)
BackgroundLayers.jsx (Estetica)

3. Gestione stato

Per la gestione dello stato ho utilizzato gli hook di React, come useState, useEffect e useRef.

Ho preferito una soluzione locale e pulita, adatta alla dimensione del progetto, senza introdurre librerie globali non necessarie. In questo modo lo stato resta sotto controllo e ogni componente gestisce soltanto ciò che gli compete.

Esempi utilizzati:

useState - Hook che serve per gestire lo stato di un componente.

Quando il dato cambia, React aggiorna la schermata.

useRef - Hook che serve per mantenere un riferimento stabile a un elemento o valore senza far ripartire il render.

useEffect - Hook che serve per eseguire codice in determinati momenti:

- quando il componente si carica
- quando cambia un dato
- quando si deve fare pulizia

4. Integrazione API

Nel progetto è stata integrata un'API per la comunicazione con il backend, cioè Laravel, sia per la gestione dei dati verso il database sia per il controllo della ventola.

Per questa parte ho utilizzato la libreria JavaScript Axios, che permette di effettuare in modo semplice le richieste tramite i principali metodi CRUD, come:

.get()

.post()

.put()

Inoltre è presente anche il controllo del token di autenticazione, necessario per consentire l'accesso alla dashboard.

5. Performance

Per quanto riguarda le performance, inizialmente avevo sviluppato un progetto molto ampio e articolato; successivamente l'ho ridimensionato attraverso un lavoro di pulizia e semplificazione.

3.3 Visualizzazione dati (6pt)

1. Grafici

I grafici rappresentano visivamente i principali parametri ambientali: temperatura, umidità, pressione, rumore, gas e illuminazione. Non hanno una funzione decorativa, ma servono a mostrare l'andamento nel tempo dei sensori in modo chiaro e immediato.

Per realizzarli ho utilizzato la libreria Recharts, in particolare i componenti AreaChart, Area, XAxis, YAxis, CartesianGrid, Tooltip e ResponsiveContainer. La parte informativa che compare quando si passa con il mouse sopra la linea del grafico è gestita proprio dal componente Tooltip, che mostra il valore del punto selezionato insieme al relativo riferimento temporale.

Ogni sensore ha un colore dedicato e fisso, definito nella configurazione dei grafici, così da mantenere coerenza visiva tra card e timeline. I grafici non mostrano l'intero storico salvato nel sistema, ma solo gli ultimi valori disponibili, in modo da rendere la lettura più pulita e utile nell'uso quotidiano.

2. Dashboard dinamiche

La dashboard non è una schermata statica, ma un'interfaccia dinamica che reagisce ai dati ricevuti dal backend. Le card dei sensori aggiornano il valore corrente, i grafici mostrano l'evoluzione recente delle misurazioni e l'orologio centrale resta sempre attivo.

I dati arrivano dalle API di Laravel in formato JSON e, quando vengono ricevuti, aggiornano lo stato dei componenti React. Di conseguenza, l'interfaccia viene renderizzata nuovamente in automatico con i nuovi valori. Anche il pannello della

ventola riflette lo stato ricevuto dal backend e aggiorna la propria interfaccia quando vengono inviate modifiche di controllo.

3. Aggiornamento realtime

L'aggiornamento avviene in tempo quasi reale tramite polling periodico ogni 10 secondi. Questa logica è gestita con setInterval all'interno di useEffect, richiamando periodicamente la funzione che recupera i dati dalla telemetria.

L'orologio centrale utilizza un secondo intervallo separato, aggiornato ogni 1 secondo, così da restare sempre sincronizzato in modo indipendente rispetto ai dati ambientali. In entrambi i casi, quando il componente viene smontato, viene eseguita la pulizia dell'intervallo tramite clearInterval, evitando chiamate inutili o possibili problemi di memoria.

Per i grafici, i dati ricevuti dal file JSON vengono trasformati in un formato compatibile con Recharts e salvati nello stato React; questo aggiornamento dello stato provoca automaticamente il ri-render della UI.

4. Leggibilità

Ho dato molta importanza alla leggibilità visiva, sia a livello grafico sia a livello di struttura dei dati mostrati. Ogni sensore è associato a un colore specifico, così da distinguerlo immediatamente nelle card e nei grafici. Le unità di misura restano sempre visibili accanto ai valori principali e sono riportate anche nella sezione dei grafici e nel Tooltip.

Per evitare grafici troppo carichi, ho limitato lo storico visualizzato agli ultimi 20 punti. In questo modo la timeline rimane leggibile, senza perdere la capacità di mostrare l'andamento recente. Le card, invece, mostrano il valore istantaneo in modo diretto, con etichetta, numero e unità ben riconoscibili.

5. Interpretazione dati

L'obiettivo non è solo mostrare numeri, ma aiutare l'utente a interpretarli. Le card mostrano il valore corrente, mentre i grafici visualizzano lo storico recente: questa combinazione permette di capire rapidamente se un parametro è stabile, in crescita o in diminuzione.

In questo modo l’utente non si limita a leggere un dato singolo, ma riesce anche a contestualizzarlo nel tempo. La dashboard diventa quindi uno strumento utile per prendere decisioni pratiche, ad esempio sul comfort domestico o sulla gestione della ventilazione.

4.5 Competenze trasversali (4pt)

1. Lavoro di gruppo

Il lavoro e’ stato suddiviso su ruoli specifici ed indipendenti l’uno dall’altro, quindi era molto individuale ma allo stesso tempo dipendente dagli altri, come gruppo non abbiamo trovato molte difficolta’ a collaborare e a chiarirci le idee.

All’inizio c’era una mancanza di comprensione, ma con adeguata pazienza e comunicazione il tutto si e’ risolto ed abbiamo continuato bene con il lavoro.

Come gruppo siamo soddisfatti di come le cose si sono svolte, poiche ogni singola cosa richiesta da chiunque e’ stata completata.

Ovviamente c’e’ qualcuno che ha messo piu impegno, pero ognuno ha fatto cio che doveva ed ha completato i propri task.

Abbiamo comunicato attraverso Whatsapp, di persona, Google Drive, Gmail e Github come principali strumenti tecnologici di comunicazione.

2. Organizzazione

Abbiamo strutturato il gruppo assegnando fin dall’inizio i ruoli.

Roman come definizione di struttura CTO,

si occupava principalmente del materiale, backend, database e pubblicazione del sito sulla rete

Inoltre si occupava dell'assegnazione degli obbiettivi.

Luka come Project Manager, si occupava del rispetto delle deadline, dei lavori doposcuola e della comunicazione tra i compagni, aiutando a mantenere ordine e continuita' nel lavoro.

Inoltre era il Frontend Team Leader.

Shaeek era la persona occupata per di piu dell'hardware, si e' occupato principalmente della parte hardware e del design del prodotto finale,

cioe della scatola finale e del modo in cui viene presentata, fornendo anche i materiali,

infine si e' occupato anche del codice implementato sul Arduino infine

De Togni si e' occupato del ruolo di UX/UI Designer, facendo da consulente per il sito, i colori, gli stili cromatici e tutto e' stato eseguito seguendo le sue direttive per il design.

Si e' inoltre occupato della parte economica e della presentazione finale.

Matteo Rossi si e' occupato della documentazione, dello studio, dei punti critici di tutto il lavoro nel complesso, definendo il linguaggio corretto per l'utilizzo ed il tracciamento dei punti finali della griglia di valutazione, e verificando che il tutto fosse completato.

3. Problem solving

Sono emerse varie problematiche, tra cui la concordanza sui ruoli e le tempistiche per trovarci tutti e 5, o comunque le persone coinvolte in presenza.

La concordanza l'abbiamo risolta con la comunicazione, imparando che e' importantissimo distinguere la responsabilita dall'amicizia e chiarire bene la comunicazione dei ruoli.

Per il tempo invece ci siamo organizzati attraverso sondaggi, recuperi online oppure sul Google Drive condiviso, dove ognuno poteva tracciare i propri progressi e comunicare i suoi avanzamenti, per cui anche i punti non chiari li abbiamo approfonditi in privato.

Devo dire che siamo stati molto svelti e che l'assegnazione iniziale dei ruoli e' servita moltissimo a creare ordine e lavoro buono.

4. Comunicazione tecnica

La comunicazione tecnica l'abbiamo gestita in vari giorni con spiegazioni, grafici, disegni e documentazione, avvenuta in ogni cartella su Google Drive e condivisa.

Abbiamo quindi risolto e organizzato ogni parte per cartella, con il proprio nome, cosi da rendere piu chiaro il contenuto, il compito svolto e la comunicazione del lavoro.

5. Autonomia

Come gruppo abbiamo creduto molto nell'autonomia e nella liberta individuale nel rendere il massimo entro le determinate deadline prestabilite.

Come anche da quello che mi riguarda, e' la politica interna di Apple, fai quando e come vuoi, basta che entro un certo tempo tu finisca il tuo lavoro.

La collaborazione c'e' stata, ma soprattutto per convenienza positiva ed appoggio reciproco, essendo anche limitati dai vincoli scolastici abbiamo fatto fatica a trovarci spesso.

Pero siamo comunque riusciti a mantenere un contatto fisso e sentirci spesso e continuamente.

PROGETTO FATTO DA

Roman Bernatskyi

Luka Ribic

Shaeek Siraj

Andrea De Togni

Matteo Rossi